Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách Khoa
Khoa Khoa học và Kỹ thuật Máy Tính

RETRIEVAL AUGMENTED GENERATION ASSIGNMENT

Đề tài:

A SIMPLE LAW CHATBOT

Giảng viên: Dr. Quản Thành Thơ

Học viên: Nguyễn Đắc Hoàng

MSHV: 2470081

Thành phố Hồ Chí Minh, 4/2026

Mục lục

1 Giới thiệu mô tả bài toán **3**
1.1 RAG là gì? . . . 3
1.2 Bài toán . . . 3
1.3 Ý nghĩa và ứng dụng . . . 3

2 Dữ liệu và chỉ số đánh giá **4**
2.1 Dữ liệu . . . 4
2.1.1 Phân tích tập Legal Documents . . . 4
2.1.2 Phân tích tập RES Dataset . . . 4
2.1.3 Nhận xét và đánh giá độ khó . . . 5
2.2 Chỉ số đánh giá . . . 5
2.2.1 Cosine Similarity . . . 5
2.2.2 Token Overlap / Jaccard Similarity . . . 5
2.2.3 BLEU Score . . . 6
2.2.4 ROUGE-L . . . 6

3 Phương pháp cải tiến RAG so với baseline **6**
3.1 Phân tích Pipeline RAG hiện tại . . . 6
3.1.1 Hạn chế của giai đoạn Indexing và Retrieval . . . 6
3.1.2 Hạn chế của giai đoạn Generation . . . 6
3.1.3 Hạn chế về môi trường triển khai . . . 7
3.2 Chiến lược cải tiến . . . 7
3.2.1 Chọn mô hình embedding . . . 7
3.2.2 Giảm độ dài chunk size . . . 7
3.2.3 Thay đổi chiến lược trong Faiss từ mặc định sang Cosine . . . 8
3.2.4 Thay đổi mô hình LLM từ Qwen1.5-1.8B sang Qwen3-1.7B . . . 8
3.2.5 Sử dụng Hybrid Search + Rerank . . . 9

4 Triển khai **9**
4.1 Xây dựng Vector Database và cấu hình Embedding . . . 10
4.2 Cấu hình Embedding và FAISS . . . 10
4.3 Chiến lược truy xuất Hybrid và tối ưu hóa tài nguyên . . . 10
4.3.1 Cơ chế Hybrid Search kết hợp Rerank . . . 10
4.3.2 Lưu trữ ngữ cảnh (Context Caching) . . . 11
4.4 Giai đoạn Sinh (Generation) với LLM . . . 11

5 Kết luận **12**
5.1 Kết quả tổng thể . . . 12
5.2 Tóm tắt kết quả đạt được . . . 13

6 Hạn chế và hướng phát triển **13**
6.1 Hạn chế . . . 13
6.2 Hướng phát triển . . . 14

Tài liệu tham khảo **15**

1 Giới thiệu mô tả bài toán

Trong bối cảnh ngày càng gia tăng lượng dữ liệu văn bản số, việc truy xuất thông tin chính xác từ các nguồn dữ liệu lớn trở nên cực kỳ quan trọng. Bài toán nghiên cứu trong báo cáo này liên quan đến việc xây dựng một hệ thống **RAG (Retrieval-Augmented Generation)**, kết hợp khả năng truy xuất thông tin từ kho dữ liệu với khả năng sinh ngôn ngữ của các mô hình ngôn ngữ lớn (LLM). Mục tiêu cụ thể là xây dựng một hệ thống **Chatbot Hỏi đáp Pháp luật** (Legal Question Answering Chatbot) thông qua việc áp dụng và tối ưu hóa kiến trúc Retrieval-Augmented Generation (RAG).

1.1 RAG là gì?

RAG (Retrieval-Augmented Generation) là một phương pháp kết hợp hai thành phần chính:

- **Retrieval (Truy xuất thông tin):** Hệ thống tìm kiếm các văn bản, đoạn thông tin liên quan từ một kho dữ liệu lớn (ví dụ: các file pháp lý, cặp câu hỏi–đáp) dựa trên truy vấn của người dùng. Các kỹ thuật phổ biến bao gồm FAISS vector search, BM25 hoặc hybrid search kết hợp nhiều phương pháp.
- **Generation (Sinh ngôn ngữ):** Một mô hình ngôn ngữ lớn (LLM) sử dụng thông tin đã được truy xuất để tạo ra câu trả lời tự nhiên, mạch lạc và phù hợp với ngữ cảnh truy vấn.

Phương pháp RAG cho phép mô hình LLM không chỉ dựa trên kiến thức nội bộ đã học, mà còn khai thác dữ liệu bên ngoài, giúp nâng cao độ chính xác, khả năng cập nhật và tính đầy đủ của câu trả lời.

1.2 Bài toán

Mục tiêu của hệ thống RAG trong báo cáo này là:

1. Truy xuất thông tin chính xác từ các **văn bản pháp lý** và **cặp câu hỏi–đáp RES** dựa trên truy vấn của người dùng.
2. Sử dụng các mô hình LLM (ví dụ: Qwen1.5-1.8B, Qwen3-1.7B) để sinh câu trả lời chi tiết, chính xác và phù hợp với ngữ cảnh.
3. So sánh hiệu quả của các chiến lược truy xuất (FAISS, BM25, hybrid search, reranking) và các mô hình LLM khác nhau dựa trên các chỉ số đánh giá như Cosine Similarity, Jaccard Similarity, Token Overlap, BLEU Score và ROUGE-L.

1.3 Ý nghĩa và ứng dụng

Hệ thống RAG được xây dựng có thể áp dụng trong nhiều lĩnh vực, đặc biệt là trong môi trường pháp lý, nơi cần truy xuất và tổng hợp thông tin chính xác từ khối lượng văn bản lớn. Các lợi ích chính bao gồm:

- Cải thiện chất lượng câu trả lời so với việc chỉ dựa vào mô hình LLM.
- Cho phép truy xuất thông tin cập nhật từ dữ liệu bên ngoài mà không cần huấn luyện lại mô hình.
- Hỗ trợ người dùng trong việc tra cứu, phân tích và đưa ra quyết định dựa trên dữ liệu pháp lý hoặc Q&A chuyên ngành.

2 Dữ liệu và chỉ số đánh giá

2.1 Dữ liệu

Bài tập cung cấp hai tập dữ liệu chính để đánh giá hệ thống RAG:

- **Legal Documents:** Tập hợp khoảng 16,440 file văn bản (`.txt`) bao phủ các lĩnh vực pháp lý khác nhau (ví dụ: dân sự, lao động, hành chính) từ nhiều cấp cơ quan khác nhau.
- **RES Dataset (Questions and Answers):** Gồm khoảng 2,745 cặp câu hỏi–đáp dùng để đánh giá hệ thống RAG. Ít nhất 100 cặp phải được thử nghiệm và số lượng sử dụng cần được báo cáo trong bài nộp.

2.1.1 Phân tích tập Legal Documents

Tập dữ liệu văn bản bao gồm tổng cộng **16,440 file .txt**. Khi phân tích độ dài từng văn bản, kết quả như sau:

- Văn bản ngắn nhất có 144 từ, trong khi văn bản dài nhất lên tới 174,697 từ. Khoảng biến thiên lớn này đặt ra thách thức trong việc lựa chọn chiến lược cắt đoạn (chunking).
- Độ dài trung bình của các văn bản là khoảng 4,188 từ với cấu trúc văn phong pháp lý chặt chẽ.

2.1.2 Phân tích tập RES Dataset

Tập dữ liệu RES gồm **2,745 cặp câu hỏi–đáp**. Trong thí nghiệm này, sử dụng một **tập con gồm 100 cặp Q&A** đầu tiên để đánh giá:

- **Độ dài câu hỏi trung bình:** 31.9 từ.
- **Độ dài câu trả lời trung bình:** 360.6 từ.

Bảng 1: Thống kê cơ bản về văn bản và cặp Q&A

Chỉ số	**Giá trị**
Tổng số file .txt	16,440
Độ dài văn bản (số từ) – nhỏ nhất	144
Độ dài văn bản (số từ) – lớn nhất	174,697
Độ dài văn bản (số từ) – trung bình	4,188.10
Tổng số cặp Q&A	2,745
Số cặp Q&A sử dụng cho thí nghiệm	100
Câu hỏi – số từ trung bình	31.90
Câu trả lời – số từ trung bình	360.59

2.1.3 Nhận xét và đánh giá độ khó

Tổng thể, dữ liệu không chỉ có sự đa dạng về độ dài mà còn chứa đựng những thách thức đặc thù về mặt nội dung:

- **Tính chất chuyên ngành cao:** Văn bản pháp luật sử dụng hệ thống thuật ngữ chuyên biệt, cấu trúc câu phức tạp. Điều này đòi hỏi mô hình Embedding phải có khả năng nắm bắt ngữ nghĩa sâu.
- **Yêu cầu về độ chính xác:** Câu trả lời trong lĩnh vực pháp lý đòi hỏi tính chính xác tuyệt đối dựa trên điều luật. Việc ảo giác (hallucination) là không thể chấp nhận.

2.2 Chỉ số đánh giá

Việc đánh giá chất lượng của các câu trả lời sinh ra ($\hat{y}$) sử dụng một tập hợp các chỉ số tương đồng so sánh câu trả lời được dự đoán ($\hat{y}$) với câu trả lời tham chiếu (chuẩn) (y).

2.2.1 Cosine Similarity

Đo lường độ tương đồng giữa các vector nhúng của câu trả lời sinh ra và câu trả lời chuẩn. Cho các vector nhúng $v_{\hat{y}}$ và v_y:

$$\text{CosineSim}(\hat{y}, y) = \frac{v_{\hat{y}} \cdot v_y}{\|v_{\hat{y}}\| \|v_y\|}$$

2.2.2 Token Overlap / Jaccard Similarity

Đo lường mức độ trùng khớp cấp độ từ giữa câu trả lời sinh ra và câu trả lời tham chiếu. Gọi $W_{\hat{y}}$ và W_y là tập hợp các token trong $\hat{y}$ và y:

$$\text{Jaccard}(\hat{y}, y) = \frac{|W_{\hat{y}} \cap W_y|}{|W_{\hat{y}} \cup W_y|} \qquad \text{TokenOverlap}(\hat{y}, y) = \frac{|W_{\hat{y}} \cap W_y|}{|W_y|}$$

2.2.3 BLEU Score

Đo lường độ chính xác n-gram của câu trả lời sinh ra so với câu trả lời tham chiếu:

$$\text{BLEU}(\hat{y}, y) = \text{BP} \cdot \frac{|W_{\hat{y}} \cap W_y|}{|W_{\hat{y}}|} \qquad \text{BP} = \begin{cases} 1, & \text{nếu } |\hat{y}| \geq |y| \\ \exp\left(1 - \frac{|y|}{|\hat{y}|}\right), & \text{nếu } |\hat{y}| < |y| \end{cases}$$

2.2.4 ROUGE-L

Đo lường độ dài của chuỗi con chung dài nhất (LCS) giữa $\hat{y}$ và y:

$$P = \frac{\text{LCS}(\hat{y}, y)}{|\hat{y}|} \qquad R = \frac{\text{LCS}(\hat{y}, y)}{|y|} \qquad \text{ROUGE-L} = \frac{2 \cdot P \cdot R}{P + R}$$

3 Phương pháp cải tiến RAG so với baseline

Việc xây dựng hệ thống RAG hiệu quả cho dữ liệu pháp lý đòi hỏi sự tối ưu hóa đồng thời ở cả hai giai đoạn: Truy xuất (Retrieval) và Sinh (Generation). Ta tiến hành cải tiến hệ thống theo từng bước có kiểm soát, dựa trên việc đánh giá định lượng hiệu suất của từng thành phần trong pipeline.

3.1 Phân tích Pipeline RAG hiện tại

Pipeline RAG cơ bản được hiện thực trong file `baseline.py`, tuân theo cấu trúc ba giai đoạn tiêu chuẩn: *Indexing*, *Retrieval* và *Generation*.

3.1.1 Hạn chế của giai đoạn Indexing và Retrieval

- **Khoảng cách đo (Distance Metric) chưa tối ưu:** Pipeline cơ bản sử dụng FAISS với khoảng cách Euclidean (L2) mặc định. Tuy nhiên, với các vector embedding đã được chuẩn hóa, Cosine Similarity thường cung cấp thước đo độ tương đồng ngữ nghĩa chính xác hơn.
- **Phụ thuộc duy nhất vào Semantic Search:** Dữ liệu pháp luật đòi hỏi độ chính xác cao về điều khoản, tên luật và cụm từ pháp lý. Việc chỉ dựa vào tìm kiếm vector có thể bỏ sót các tài liệu quan trọng nếu vector ngữ nghĩa bị nhiễu.
- **Lựa chọn Chunking và Embedding Model chưa tối ưu:** Các tham số như kích thước chunk size và mô hình embedding cần được đánh giá lại để đảm bảo tính phù hợp với đặc thù ngôn ngữ pháp lý tiếng Việt.

3.1.2 Hạn chế của giai đoạn Generation

- **Chất lượng đầu vào của LLM:** Ngữ cảnh nhiễu, thiếu thông tin hoặc chứa mâu thuẫn có thể dẫn đến hiện tượng suy diễn sai (hallucination).

- **Prompt Engineering:** Prompt template cơ bản chưa đủ mạnh để đảm bảo khả năng định hướng (grounding), khiến mô hình có thể trả lời vượt ra ngoài phạm vi thông tin được cung cấp.

3.1.3 Hạn chế về môi trường triển khai

- **Giới hạn tài nguyên trên Google Colab:** Mô hình yêu cầu tải đồng thời Vector Store (FAISS), Embedding Model và LLM, dễ dẫn đến lỗi *Out of Memory* (OOM).

3.2 Chiến lược cải tiến

Dựa trên các phân tích trên, ta xây dựng chiến lược cải tiến tập trung vào ba trụ cột chính:

- **Tối ưu hóa tham số Indexing:** Xác định Chunk Size và khoảng cách đo lường phù hợp nhất.
- **Tăng cường truy xuất nâng cao:** Áp dụng Hybrid Search kết hợp FAISS và BM25, cùng với bước Reranking.
- **Tối ưu hóa triển khai:** Thực hiện chiến lược *Context Pre-computation* nhằm giải quyết vấn đề giới hạn tài nguyên.

3.2.1 Chọn mô hình embedding

Tiến hành so sánh các mô hình embedding dựa trên Cosine Similarity với 10 cặp Q&A đầu tiên trong `res.csv`:

Bảng 2: So sánh Mean Cosine Similarity giữa các mô hình embedding

Mô hình	Mean Cosine Similarity
`BAAI/bge-m3`	0.876006
`keepitreal/vietnamese-sbert`	0.858670
`huyydangg/DEk21_hcmute_embedding`	0.812371
`AITeamVN/Vietnamese_Embedding`	0.782572

Mô hình `BAAI/bge-m3` có kết quả tốt nhất (0.876006) nhưng yêu cầu tài nguyên tính toán lớn, không thể chạy hiệu quả trên Colab cùng LLM. Mô hình `keepitreal/vietnamese-sbert` (0.858670) nhẹ hơn và ổn định hơn, phù hợp với môi trường tài nguyên hạn chế. Do đó, `keepitreal/vietnamese-sbert` được chọn làm mô hình embedding chính.

3.2.2 Giảm độ dài chunk size

Thử nghiệm giảm chunk size từ 2000 xuống 1024:

Bảng 3: So sánh các chỉ số đánh giá theo độ dài chunk size

Chỉ số	Chunk size = 1024	Chunk size = 2000 (baseline)
Cosine Similarity	0.6871	0.6867
Jaccard Similarity	0.2335	0.1961
Token Overlap	0.6716	**0.7434**
BLEU Score	0.1615	0.1041
ROUGE-L	0.1215	0.0899

Chunk size = 1024 cải thiện hầu hết các chỉ số, ngoại trừ Token Overlap giảm nhẹ do chunk nhỏ hơn làm giảm số lượng token chung giữa các đoạn văn dài. Do đó, **chunk size = 1024** được chọn cho các thí nghiệm tiếp theo.

3.2.3 Thay đổi chiến lược trong Faiss từ mặc định sang Cosine

Các mô hình embedding được huấn luyện để so sánh độ tương đồng dựa trên Cosine Similarity, do đó việc tiếp tục sử dụng khoảng cách Euclidean mặc định sẽ không phản ánh chính xác mức độ tương đồng ngữ nghĩa. Thay đổi được thực hiện như sau:

Listing 1: Thay đổi distance strategy trong FAISS sang Cosine

```
vectorstore = FAISS.from_documents(
    documents=batch_docs,
    embedding=embeddings,
    distance_strategy=DistanceStrategy.COSINE  # <- bo sung
)
```

Bảng 4: Kết quả đánh giá khi sử dụng Cosine distance trong Faiss (chunk size = 1024)

Chỉ số đánh giá	Giá trị
Cosine Similarity	0.6980
Jaccard Similarity	0.2352
Token Overlap	0.6683
BLEU Score	0.1635
ROUGE-L	0.1232

3.2.4 Thay đổi mô hình LLM từ Qwen1.5-1.8B sang Qwen3-1.7B

Mô hình `Qwen1.5-1.8B` tồn tại nhiều hạn chế trong việc sinh văn bản tiếng Việt: các câu trả lời thường thiếu tự nhiên, sai ngữ pháp, hoặc xuất hiện hiện tượng pha trộn giữa tiếng Việt và tiếng Trung. Do đó, quyết định thay thế bằng `Qwen3-1.7B` – phiên bản được huấn luyện với tập dữ liệu đa ngôn ngữ phong phú hơn và hỗ trợ tiếng Việt tốt hơn.

Bảng 5: Kết quả Qwen3-1.7B (Cosine distance, chunk size = 1024)

Chỉ số	Qwen3-1.7B
Cosine Similarity	0.6980
Jaccard Similarity	0.2400
Token Overlap	0.7007
BLEU Score	0.1664
ROUGE-L	0.1274

3.2.5 Sử dụng Hybrid Search + Rerank

Hybrid Search kết hợp điểm số từ hai phương pháp:

- **Dense Retrieval (FAISS):** Ưu tiên sự tương đồng về **ngữ nghĩa**.
- **Lexical Retrieval (BM25):** Ưu tiên sự tương đồng về **từ khóa**.

Điểm số cuối cùng được tính theo công thức:

$$\text{Score}_{\text{Hybrid}} = \alpha \cdot \text{Score}_{\text{Dense}} + (1-\alpha) \cdot \text{Score}_{\text{Lexical}}$$

Sau đó, bước **Rerank** dùng mô hình Cross-Encoder (`BAAI/bge-reranker-large`) để tinh chỉnh thứ hạng, loại bỏ các tài liệu nhiễu trước khi đưa vào LLM.

Bảng 6: Kết quả Qwen3-1.7B với Hybrid Search và Hybrid Search + Rerank

Chỉ số	Hybrid Search	Hybrid Search + Rerank
Cosine Similarity	0.6993	**0.7026**
Jaccard Similarity	0.2413	**0.2447**
Token Overlap	0.7149	**0.7173**
BLEU Score	0.1685	**0.1691**
ROUGE-L	**0.1333**	0.1310

Việc bổ sung rerank giúp cải thiện đáng kể các chỉ số, đặc biệt Cosine Similarity (từ 0.6993 lên 0.7026) và Token Overlap (từ 0.7149 lên 0.7173), chứng tỏ ngữ cảnh được truy xuất có tính tương đồng ngữ nghĩa cao hơn với câu trả lời chuẩn.

4 Triển khai

Quá trình hiện thực hệ thống RAG được tiến hành trên môi trường Google Colab. Do hạn chế về tài nguyên phần cứng (RAM và VRAM) khi phải đồng thời tải cả Vector Database, Embedding Model và LLM, đề xuất chia quá trình triển khai thành hai giai đoạn độc lập: (1) Xây dựng dữ liệu và Truy xuất ngữ cảnh và (2) Sinh câu trả lời.

4.1 Xây dựng Vector Database và cấu hình Embedding

Giai đoạn đầu tiên là chuyển đổi dữ liệu văn bản pháp luật thành các vector đặc trưng sử dụng LangChain và FAISS:

- **Xử lý dữ liệu:** Chunking với `chunk_size=1024`.
- **Embedding Model:** `keepitreal/vietnamese-sbert` với cấu hình chuẩn hóa vector.
- **Vector Store:** FAISS với `DistanceStrategy.COSINE`.

4.2 Cấu hình Embedding và FAISS

Listing 2: Cấu hình Embedding và khởi tạo FAISS với Cosine Strategy

```
encode_kwargs = {"normalize_embeddings": True}

embeddings = HuggingFaceEmbeddings(
    model_name="keepitreal/vietnamese-sbert",
    model_kwargs={"device": "cuda"},
    encode_kwargs=encode_kwargs
)

vectorstore = FAISS.from_documents(
    documents=batch_docs,
    embedding=embeddings,
    distance_strategy=DistanceStrategy.COSINE
)
```

4.3 Chiến lược truy xuất Hybrid và tối ưu hóa tài nguyên

Thay vì truy xuất trực tiếp trong lúc sinh câu trả lời (Online Retrieval), thực hiện *Pre-computation* ngữ cảnh cho toàn bộ tập dữ liệu kiểm thử (RES Dataset) trước, nhằm tránh tình trạng OOM khi tải LLM.

4.3.1 Cơ chế Hybrid Search kết hợp Rerank

Xây dựng lớp `HybridFAISSBM25Searcher` kết hợp Dense Retrieval (FAISS) và Lexical Retrieval (BM25), sau đó rerank bằng `BAAI/bge-reranker-large`:

Listing 3: Lớp HybridFAISSBM25Searcher

```
class HybridFAISSBM25Searcher:

    def _hybrid_search(self, query, top_k, dense_k, alpha):
        dense_results = self.faiss_db.similarity_search_with_score
            (
             query, k=dense_k
        )
```

```
        bm25_scores = self.bm25.get_scores(self.fast_tokenize(
            query))
        # score = alpha * dense + (1 - alpha) * lexical
        return results[:top_k]

    def _rerank(self, query, docs, top_k):
        pairs = [(query, doc.page_content) for doc in docs]
        scores = self.reranker.predict(pairs)
        return ranked[:top_k]
```

4.3.2 Lưu trữ ngữ cảnh (Context Caching)

Truy xuất ngữ cảnh cho toàn bộ câu hỏi và lưu vào file trung gian `res_with_context.csv` với các cột:

- `context_baseline`: Ngữ cảnh từ truy xuất cơ bản.
- `context_hybrid`: Ngữ cảnh từ FAISS + BM25.
- `context_hybrid_rerank`: Ngữ cảnh sau bước Reranker.

Listing 4: Lưu trữ ngữ cảnh vào file CSV trung gian

```
contexts_rerank = []
for question in tqdm(df["Cau Hoi"], desc="Building contexts"):
    results = searcher.search(
        query=question, use_rerank=True, rerank_k=10
    )
    contexts_rerank.append(build_context(results))

df["context_hybrid_rerank"] = contexts_rerank
df.to_csv("res_with_context.csv", index=False)
```

4.4 Giai đoạn Sinh (Generation) với LLM

Sau khi giải phóng bộ nhớ khỏi Vector Store và Embedding Model, tải mô hình `Qwen/Qwen3-1.7B` để sinh câu trả lời từ ngữ cảnh đã được chuẩn bị sẵn:

Listing 5: Khởi tạo LLM và hàm RAG query với Prompt Grounding

```
llm = HuggingFacePipeline.from_model_id(
    model_id="Qwen/Qwen3-1.7B",
    task="text-generation",
    device=0,
    model_kwargs={"temperature": 0, "max_length": 1024}
)

def rag_query(question, context):
```

```
    prompt = f"""Ban la mot tro ly thong minh.
Tra loi cau hoi dua tren ngu canh sau.
Neu khong du thong tin, hay noi ro dieu do,
khong duoc tu bia dat.

Ngu canh: {context}
Cau hoi: {question}
Tra loi ngan gon va chinh xac nhat co the:"""

    output = llm.generate([prompt], max_new_tokens=256)
    return output.generations[0][0].text
```

5 Kết luận

Báo cáo này đã trình bày chi tiết quá trình xây dựng, tối ưu hóa và đánh giá một hệ thống RAG dành cho lĩnh vực pháp luật Việt Nam. Hệ thống được phát triển nhằm cung cấp câu trả lời chính xác, dựa trên ngữ cảnh từ kho văn bản luật lớn, đồng thời thiết lập một quy trình triển khai ổn định trên môi trường tài nguyên hạn chế.

5.1 Kết quả tổng thể

Bảng 7: So sánh chỉ số đánh giá: Baseline vs. Hệ thống tối ưu hóa

Chỉ số	Hệ thống ban đầu	Hệ thống tối ưu hóa
Cosine Similarity	0.6871	**0.7026**
Jaccard Similarity	0.2335	**0.2447**
Token Overlap	0.6716	**0.7173**
BLEU Score	0.1615	**0.1691**
ROUGE-L	0.1215	**0.1310**

Hệ thống tối ưu hóa (Hybrid Search + Rerank, Cosine Distance, chunk size 1024, Qwen3-1.7B) cải thiện đồng bộ trên tất cả các chỉ số so với baseline (Qwen1.5-1.8B, chunk size 2000, L2 Distance), thể hiện sự nâng cao chất lượng câu trả lời một cách toàn diện.

- **Cosine Similarity** (0.6871 → 0.7026): Ngữ cảnh truy hồi phù hợp hơn nhờ Hybrid Search + Rerank; mô hình Qwen3-1.7B hiểu và diễn đạt đúng ý nghĩa câu hỏi tốt hơn.
- **Jaccard Similarity** (0.2335 → 0.2447): Câu trả lời chứa nhiều thuật ngữ và từ khóa quan trọng giống với câu trả lời chuẩn hơn.
- **Token Overlap** (0.6716 → 0.7173): Chỉ số tăng mạnh nhất – hệ thống truy hồi được đoạn văn chứa thông tin đúng trọng tâm, giảm tình trạng hallucination.

- **BLEU Score** (0.1615 → 0.1691): Cấu trúc câu và cách diễn đạt gần hơn với câu trả lời chuẩn.
- **ROUGE-L** (0.1215 → 0.1310): Câu trả lời có bố cục logic và trình tự nội dung gần với câu trả lời chuẩn hơn.

5.2 Tóm tắt kết quả đạt được

1. **Tối ưu hóa truy xuất:**
   - Xác định chiến lược Chunking tối ưu (`chunk_size=1024`) và Cosine Similarity để tăng độ chính xác truy xuất ngữ nghĩa.
   - Triển khai thành công **Hybrid Search (FAISS + BM25)** kết hợp **Rerank** (`BAAI/bge-reranker-large`).
2. **Cải thiện chất lượng sinh:**
   - Giải quyết vấn đề sinh văn bản tiếng Việt kém chất lượng bằng cách chuyển sang `Qwen3-1.7B`.
3. **Giải pháp triển khai thực tế:**
   - Hiện thực chiến lược **Context Pre-computation** (tách biệt Retrieval và Generation) khắc phục triệt để vấn đề giới hạn tài nguyên Google Colab.

6 Hạn chế và hướng phát triển

Mặc dù hệ thống RAG đã được tối ưu hóa đáng kể, hệ thống vẫn còn tồn tại một số hạn chế và tiềm năng cải thiện trong tương lai.

6.1 Hạn chế

1. **Mô hình LLM kích thước nhỏ:** Việc sử dụng `Qwen3-1.7B` là giải pháp bắt buộc để hoạt động trên tài nguyên Colab. Mô hình nhỏ này có thể hạn chế khả năng suy luận phức tạp và tổng hợp thông tin từ nhiều đoạn ngữ cảnh dài.
2. **Giới hạn của phương pháp đánh giá:** Các metrics tự động như Cosine Similarity, ROUGE-L và BLEU không thể đánh giá toàn diện tính chính xác của thông tin, đặc biệt trong lĩnh vực pháp lý.
3. **Cấu hình RAG tĩnh (Offline RAG):** Chiến lược Context Pre-computation không phải là giải pháp thời gian thực. Hệ thống không thể xử lý truy vấn mới ngay lập tức nếu dữ liệu cơ sở thay đổi.
4. **Độ nhạy của tham số Hybrid Search:** Hiệu suất phụ thuộc nhiều vào việc lựa chọn hệ số α. Việc tinh chỉnh thủ công α có thể không đảm bảo tính tối ưu trên toàn bộ tập dữ liệu.

6.2 Hướng phát triển

1. **Tối ưu hóa truy vấn nâng cao:**
   - **Query Rewriting:** Sử dụng LLM để viết lại câu hỏi ban đầu trước khi gửi đến Vector Store, giúp truy xuất tài liệu chính xác hơn.
     Ví dụ: "Tôi bị chậm lương thì làm thế nào?" → "Điều 94 Bộ luật Lao động 2019 quy định về trách nhiệm chi trả lương khi bị chậm."
   - **Phân rã truy vấn:** LLM phân rã câu hỏi phức tạp thành nhiều câu hỏi con độc lập, giúp giải quyết các yêu cầu suy luận pháp lý đa bước.
2. **Tối ưu hóa cấu trúc ngữ cảnh – RAPTOR:** Kiến trúc RAPTOR tạo ra các chunks ở nhiều cấp độ, sử dụng LLM để tóm tắt các đoạn nhỏ thành các đoạn trừu tượng hơn, phù hợp với tài liệu pháp lý có cấu trúc phân cấp.
3. **Nâng cấp mô hình:**
   - **Embedding chuyên ngành:** Khám phá các mô hình như `BKAI/vietnamese-large-bge-m3` hoặc fine-tune `keepitreal/vietnamese-sbert` trên tập dữ liệu pháp luật được gán nhãn.
   - **LLM lớn hơn:** Chuyển sang các mô hình lớn hơn như PhoGPT hoặc `Qwen 7B` (đã lượng tử hóa) để nâng cao khả năng suy luận phức tạp và xử lý ngữ cảnh dài.

Tài liệu tham khảo

THIS IS THE END OF THE REPORT